Технічне завдання — «Стандартотворець»

> ### ⚠️ Статус документа: ЗАМОРОЖЕНА БАЗА (травень 2026)
>
> Це **початкове ТЗ** — що замовляли на старті. Воно свідомо **не правиться** під реалізацію: його цінність у тому, щоб можна було звірити зроблене з початковими вимогами.
>
> **Поточний стан системи описано в `docs/ARCHITECTURE.md`** — читай його, якщо треба зрозуміти, як усе влаштовано зараз.
>
> **Головні розбіжності з реалізацією** (станом на 2026-08-06):
>
> | Розділ ТЗ | Що змінилось |
> |---|---|
> | §4 Доменна модель | Схема виросла до 26 моделей + 20 enum (882 рядки `schema.prisma`). Додалось: `StandardSuggestion`, `SuggestionReaction`, `InlineComment` (+ `Reply`), `TaskChecklistItem`, `StandardStatusHistory`, `RolePermission`, `InviteToken`, `SystemSettings` |
> | §4 `Standard` | Додано поетапний план (5 етапів × дедлайн + відмітка), метадані Програми (`partProgram` / `programNumber` / `indeks` / `oldTitle`), `isTaskTemplate`, тіло документа (`bodyText`) |
> | §4 `Document` | Додано заморожування: `lockedAt`, `lockedByVotingId`; типи перебудовано (`DRAFT_STANDARD` → `STANDARD`, `FINAL` прибрано) |
> | §4 `Voting` | Додано `seqNumber`, `eligibleAtClose`, `documentId` |
> | §3 Ролі | Дефолти винесено в матрицю з DB-перевизначенням (`RolePermission`, `/admin/permissions`) — у ТЗ права були жорстко зашиті |
> | §5/§6 Фази | Обидві фази реалізовано; понад ТЗ додано: спільну роботу з текстом (пропозиції + інлайн-коментарі), ШІ-чернетку протоколу, PDF/Word-експорт, iCal, глобальний пошук, undo через аудит, шаблони задач |
> | §9 Чого немає | Актуальний перелік — `docs/ARCHITECTURE.md` §16 |

> Платформа управління життєвим циклом стандартів у робочих групах. Версія документа: 1.0 · Дата: 2026-05-18.

Документ описує систему так, ніби її ще немає. Розділ **«База»** — мінімум, без якого система не запрацює і команда не зможе вести справи спільно. Розділ **«Покращення»** — функціонал який різко піднімає зручність, але без нього базовий процес теж працює.

---

1. Огляд

1.1 Призначення

Облік стандартизаційної діяльності робочих груп (РГ) органу. Заміна типового сценарію «купа Excel + Outlook + Word + папка на мережевому диску» на єдину систему де:

- кожен стандарт має офіційний lifecycle із дедлайнами по етапах
- кожне засідання має формальний протокол з ПОРЯДКОМ ДЕННИМ / СЛУХАЛИ / ВИРІШИЛИ
- кожна дія обкутана автоматичними сповіщеннями і повним audit-trail
- керівництво бачить агреговану картину готовності плану

1.2 Цільова аудиторія

~50–200 користувачів з 4–10 РГ. Військово-цивільна організація, тому є військові звання і посади.

1.3 Ключові метрики успіху

- ≥ 90% засідань мають оцифрований протокол протягом 24 год після проведення
- ≥ 80% етапів стандартів закриваються в строк завдяки нагадуванням
- Час підготовки місячного звіту для керівництва зменшується з днів до 0 (генерується автоматично)

---

2. Глосарій

Термін	Значення
РГ	Робоча група. Атомарна одиниця організації. Має керівника, заступника, секретаря, членів.
Стандарт	Документ що проходить життєвий цикл (Чернетка → На розгляді → Голосування → Прийнятий/Відхилений → Архів).
Етап	Контрольна віха в розробці стандарту: ТЗ → Проєкт → Відгуки → Перевірка → Остаточна редакція. Кожен має планову і фактичну дату виконання.
Засідання	Захід РГ з фіксованою датою, форматом (Онлайн/Офлайн/Гібрид), порядком денним.
Протокол	Офіційний документ за результатами засідання. Має 3 розділи: ПОРЯДОК ДЕННИЙ, СЛУХАЛИ/ВИСТУПИЛИ, ВИРІШИЛИ.
Кворум	Більш ніж 50% статутних членів РГ підтвердили присутність. Секретарі та гості не враховуються.
Голосування	Формальне голосування «за/проти/утримався» за стандарт.
Дедлайн етапу	Планова дата виконання етапу. Прострочка тригерить сповіщення.

---

3. Ролі та права доступу

3.1 Глобальні ролі (`GlobalRole`)

- **ADMIN** — повний доступ до системних налаштувань, керування користувачами, видалення сутностей.
- **DIRECTOR** — read-only по всіх РГ, доступ до агрегованої аналітики. Не керує РГ.
- **USER** — звичайний користувач. Бачить тільки РГ де він член.

3.2 Ролі в РГ (`WorkingGroupRole`)

- **LEADER** (керівник РГ) — повне керування РГ, її стандартами, засіданнями.
- **DEPUTY** (заступник) — те саме що керівник.
- **SECRETARY** (секретар) — створює засідання, веде протоколи, керує учасниками. НЕ враховується в кворумі.
- **MEMBER** (член) — голосує, виконує завдання, коментує.
- **GUEST** (гість) — read-only член, не голосує, не враховується в кворумі.

3.3 Матриця прав (скорочено)

Описати в `src/lib/rbac.ts` як функцію `can(userCtx, action, scope?)`. Базові дії:

Дія	ADMIN	DIRECTOR	LEADER	DEPUTY	SECRETARY	MEMBER	GUEST
Створити стандарт у своїй РГ	✓	✗	✓	✓	✓	✗	✗
Змінити статус стандарту	✓	✗	✓	✓	✓	✗	✗
Підтвердити етап стандарту	✓	✗	✓	✓	✓	✗	✗
Створити засідання	✓	✗	✓	✓	✓	✗	✗
Завантажити/редагувати протокол	✓	✗	✓	✓	✓	✗	✗
Керувати присутністю інших	✓	✓	✓	✓	✓	✗	✗
Підтвердити власну присутність	✓	✓	✓	✓	✓	✓	✗
Відкрити голосування	✓	✗	✓	✓	✓	✗	✗
Голосувати	✓	✗	✓	✓	✓	✓	✗
Коментувати	✓	✓	✓	✓	✓	✓	✗
Бачити audit-log своєї РГ	✓	✓	✓	✓	✓	✓	✗
Запросити нового користувача в РГ	✓	✗	✓	✓	✓	✗	✗
Архівувати РГ	✓	✗	✗	✗	✗	✗	✗

4. Доменна модель (ORM)

Postgres 18+, Prisma 5+. Ключові таблиці:

```
model User {
  id           String   @id @default(cuid())
  email        String   @unique
  passwordHash String
  name         String
  rank         MilitaryRank @default(CIVILIAN)
  position     String?
  organization Organization @default(OTHER)
  globalRole   GlobalRole   @default(USER)
  isActive     Boolean      @default(true)
  notifyInApp  Boolean      @default(true)
  notifyEmail  Boolean      @default(true)
  phone        String?
  avatarUrl    String?
  memberships  WorkingGroupMember[]
  // …
}

model WorkingGroup {
  id          String   @id @default(cuid())
  code        String   @unique     // e.g. "РГ №4"
  name        String                // human description
  description String?
  color       String   @default("#1A56DB")
  isArchived  Boolean  @default(false)
  members     WorkingGroupMember[]
  standards   Standard[]
  meetings    Meeting[]
}

model WorkingGroupMember {
  workingGroupId String
  userId         String
  role           WorkingGroupRole
  joinedAt       DateTime @default(now())
  @@id([workingGroupId, userId])
}

model Standard {
  id             String   @id @default(cuid())
  code           String   @unique          // e.g. "ДСТУ 7.1:2026"
  title          String
  description    String?
  workingGroupId  String
  status         StandardStatus @default(DRAFT)
  isoAnalog      String?
```

```
  category        String?
  responsibleId   String?
  // Program-plan stage fields (5 stages × 2 timestamps each):
  techSpecDueDate       DateTime?
  techSpecCompletedAt   DateTime?
  draftDueDate          DateTime?
  draftCompletedAt      DateTime?
  feedbackDueDate       DateTime?
  feedbackCompletedAt   DateTime?
  techReviewDueDate     DateTime?
  techReviewCompletedAt DateTime?
  finalDueDate          DateTime?
  finalCompletedAt      DateTime?
  currentStage          StandardStage @default(TECH_SPEC)
  indeks                String?       // ДСТУ-index for program-plan items
  programNumber         Int?          // order in program plan
  // …
}

model Meeting {
  id              String   @id @default(cuid())
  workingGroupId  String
  title           String
  format          MeetingFormat @default(OFFLINE)  // ONLINE/OFFLINE/HYBRID
  location        String?
  startAt         DateTime
  durationMins    Int      @default(60)
  agendaText      String?
  minutesText     String?
  status          MeetingStatus @default(PLANNED)  // PLANNED/IN_PROGRESS/COMPLETED/CANCELLED
  chairmanId      String?
  protocolNumber  Int?
  createdById     String
  agendaItems     AgendaItem[]
  attendances     Attendance[]
}

model AgendaItem {
  id            String   @id @default(cuid())
  meetingId     String
  section       AgendaItemSection @default(AGENDA)  // AGENDA/HEARD/DECISION
  order         Int
  title         String
  speakerId     String?
  heardText     String?
  discussionText String?
  decisionText  String?
```

```
  deadline      DateTime?
  responsibleId String?
}

model Attendance {
  meetingId String
  userId    String
  status    AttendanceStatus @default(PENDING)     // PENDING/CONFIRMED/DECLINED
  note      String?
  @@id([meetingId, userId])
}

model Task {
  id            String   @id @default(cuid())
  standardId    String
  title         String
  description   String?
  status        TaskStatus @default(OPEN)
  priority      TaskPriority @default(MEDIUM)
  dueDate       DateTime?
  assigneeId    String?
  createdById   String
}

model Voting {
  id            String   @id @default(cuid())
  standardId    String
  title         String
  description   String?
  status        VotingStatus @default(OPEN)
  deadline      DateTime?
  votes         Vote[]
}

model Vote {
  votingId  String
  userId    String
  choice    VoteChoice         // FOR/AGAINST/ABSTAIN
  votedAt   DateTime @default(now())
  @@id([votingId, userId])
}

model Comment {
  id          String   @id @default(cuid())
  standardId  String
  authorId    String
  parentId    String?            // 2 level nesting only
```

```
  body        String
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
}

model Notification {
  id        String   @id @default(cuid())
  userId    String
  type      NotificationType
  title     String
  body      String?
  link      String?
  read      Boolean  @default(false)
  createdAt DateTime @default(now())
}

model ActivityLog {
  id        String   @id @default(cuid())
  userId    String?
  action    AuditAction          // CREATE/UPDATE/DELETE/STATUS_CHANGE/LOGIN
  entity    String                // "Standard" / "Meeting" / etc.
  entityId  String
  before    Json?
  after     Json?
  note      String?
  createdAt DateTime @default(now())
}

model SystemSettings {
  id Int @id @default(1)
  // Channel-level toggles
  channelInApp Boolean @default(true)
  channelEmail Boolean @default(true)
  // Per-event toggles + lead times (see Notifications section)
  meetingInviteOnCreate    Boolean @default(true)
  meetingChangeNotify      Boolean @default(true)
  meetingRemindLead1Hours  Int     @default(24)
  meetingRemindLead2Hours  Int?    @default(1)
  taskAssignNotify         Boolean @default(true)
  taskDeadlineLeadHours    Int     @default(24)
  taskOverdueNotify        Boolean @default(true)
  taskCompleteNotify       Boolean @default(true)
  voteOpenedNotify         Boolean @default(true)
  voteClosingLeadHours     Int     @default(24)
  voteClosedNotify         Boolean @default(true)
  standardStatusNotify     Boolean @default(true)
  commentMentionNotify     Boolean @default(true)
```

```
  stageDueSoonNotify       Boolean @default(true)
  stageDueLeadDays1        Int     @default(7)
  stageDueLeadDays2        Int     @default(1)
  stageOverdueNotify       Boolean @default(true)
  stageCompletedNotify     Boolean @default(true)
  weeklyDigestEnabled      Boolean @default(true)
  attendanceDeclinedNotify Boolean @default(true)
  protocolPublishedNotify  Boolean @default(true)
}
```

Усі мутації **зобов'язані** писати рядок у `ActivityLog`. Покриття перевіряється статичним аналізом (`pnpm test:audit-coverage`) і блокується в CI.

---

5. БАЗА (Phase 1) — мінімум для запуску

Без цього система не дає цінності. Працює і збирає дані, але без полісів і покращень.

5.1 Аутентифікація і профіль

5.1.1 Логін — email + пароль. Hash bcrypt rounds=12. Cookie-based session (NextAuth Credentials provider, JWT, 30 днів).

5.1.2 Запрошення нового користувача — ADMIN/LEADER генерує одноразовий invite token, надсилає email з посиланням `/invite/<token>` де новий користувач задає ПІБ, звання, посаду, пароль. Token expires через 7 днів.

5.1.3 Профіль (`/profile`) — користувач редагує ПІБ, email, телефон, тумблери `notifyInApp` / `notifyEmail`. Бачить список своїх РГ з ролями.

5.1.4 Скидання пароля — поки що не обов'язково. ADMIN може скинути через `/admin/users`. (Самостійний reset через email — у Phase 2.)

5.2 Робочі групи

5.2.1 Список РГ (`/working-groups`) — картки з кодом, назвою, кількістю членів і стандартів. Архівовані відображаються в окремій вкладці.

5.2.2 Сторінка РГ (`/working-groups/[id]`) — 5 вкладок: Інформація, Учасники, Стандарти, Засідання, Документи.

5.2.3 Керування учасниками — LEADER/SECRETARY запрошує користувачів. Зміна ролі (`changeMemberRole`). Видалення.

5.2.4 Створення РГ — тільки ADMIN. Унікальний `code`, назва, опис, колір.

5.3 Стандарти

5.3.1 CRUD стандарту — створення (`/standards/new`), редагування, видалення (тільки ADMIN, фізичне).

5.3.2 Лінійний lifecycle статусів (`StandardStatus`):

```
DRAFT ──► IN_REVIEW ──► VOTING ──► ADOPTED ──► ARCHIVED
                                  └──► REJECTED ──► (DRAFT or ARCHIVED)
```

Кожна зміна статусу пише в `StandardStatusHistory` + ActivityLog + сповіщення усім членам РГ.

5.3.3 5 етапів плану — для кожного стандарту з програмного плану зберігається `<stage>DueDate` (планова дата) і `<stage>CompletedAt` (фактична). Етапи: `TECH_SPEC` , `DRAFTING` , `FEEDBACK` , `TECH_REVIEW` , `FINALIZATION` . Підтвердження виконання — кліком LEADER/DEPUTY/SECRETARY. Дату виконання можна задати руками (не тільки `now()`).

5.3.4 Список стандартів (`/standards`) — таблиця з фільтрами:

- Пошук (мін 2 символи)
- Multi-select WG
- Multi-select статус
- Сортування: код, РГ, статус, відповідальний, дедлайн, найближчий етап

Стандартний пагінатор, 20 на сторінку. Фільтри в `localStorage` (запам'ятовуються per browser).

5.3.5 Деталь стандарту (`/standards/[id]`) — 6 вкладок: Документи, Обговорення, Завдання, Учасники РГ, Голосування, Історія. Опис завжди над вкладками. Stepper «Поетапний план виконання» — згортний, стан запам'ятовується (per user, per standard).

5.4 Засідання + Протоколи

5.4.1 Календар (`/meetings`) — місяць/тиждень/список. Кольорові пілюлі по РГ. Multi-WG фільтр.

5.4.2 Створення засідання — поля: РГ (обов'язково), тема, дата+час, тривалість, формат (ONLINE/OFFLINE/HYBRID, дефолт OFFLINE), локація, порядок денний (текстовий), головуючий (за замовчуванням = керівник РГ). При створенні автоматично:

- Створюються `Attendance` рядки для всіх членів РГ зі статусом `PENDING`
- Шлеться сповіщення всім членам

5.4.3 Деталь засідання — інфо-карта 3×2 (Головуючий | Дата | Тривалість \ Кворум | Формат | Організатор), список присутніх з кнопками статусу для привілейованих, картка-превью протоколу.

5.4.4 Редактор протоколу (`/meetings/[id]/protocol`) — основна сторінка справа: вкладки **Текст / ПОРЯДОК ДЕННИЙ / СЛУХАЛИ / ВИРІШИЛИ**. Зліва вузький сайдбар учасників (260px) з ролями і кнопками статусу. Кожна секція — окремий список пунктів. Один єдиний кнопка «Зберегти все» (єдиний batch save для усіх dirty-пунктів).

5.4.5 Експорт протоколу — `GET /api/meetings/[id]/protocol.docx` (Word) і `/protocol` (PDF). Word збирається через `docx` пакет; PDF через `@react-pdf/renderer`. Шапка з реквізитами (ПРОТОКОЛ № N//<рік>, дата, місто), потім ПІБ головуючого/секретаря, потім порядок денний, СЛУХАЛИ, ВИРІШИЛИ, підписи.

5.4.6 Номер протоколу — присвоюється кліком «Присвоїти №» у редакторі. Послідовний per WG per year. Цей крок офіційно «публікує» протокол.

5.5 Завдання

5.5.1 CRUD задач — належать стандарту, мають виконавця, дедлайн, пріоритет, статус. Прості (без чек-листів).

5.5.2 Сторінка завдань (`/tasks`) — дерево «РГ → Стандарт → Завдання» зліва, список справа. Фільтри: Всі / Відкриті / Виконані / Мої.

5.6 Голосування

5.6.1 Відкриття голосування — LEADER/DEPUTY/SECRETARY створює голосування на стандарті. Дедлайн необов'язковий.

5.6.2 Голос — `FOR/AGAINST/ABSTAIN` . Один голос на користувача, можна змінити поки голосування відкрите.

5.6.3 Закриття — кнопкою або по дедлайну (Phase 1: тільки кнопкою). При закритті фіксується результат, шлеться сповіщення.

5.7 Коментарі / Обговорення

5.7.1 Коментар під стандартом. Дозволено 2 рівні (root + reply).

5.7.2 `/discussions` — фід коментарів за всіма доступними стандартами, з відмічанням «нові з мого минулого візиту» (timestamp в `localStorage`).

5.8 Сповіщення

Центральний диспетчер у `src/server/notify.ts` . Кожна подія має свій тумблер у `SystemSettings` . Два канали: in-app (рядок у `Notification`) + email (Resend або SMTP).

База — мінімум подій:

Подія	Кому	Канал	Тригер
Створено засідання	усі члени РГ	in-app + email	`meeting.create`
Засідання змінилось	усі члени РГ	in-app + email	`meeting.update` (дата/agenda)
Призначено завдання	виконавець	in-app + email	`task.create` / `task.update`
Прострочено завдання	виконавець + автор	in-app + email	cron щогодини
Відкрито голосування	усі члени РГ	in-app + email	`vote.openVoting`
Закрито голосування	усі члени РГ	in-app	`vote.closeVoting`
Змінився статус стандарту	усі члени РГ	in-app	`standard.changeStatus`
Дедлайн етапу за N днів	усі члени РГ + керівництво	in-app	cron 09:00 Київ
Етап прострочено	керівництво РГ + DIRECTOR	in-app + email	cron 09:00 Київ, одноразово
Етап виконано	керівництво + DIRECTOR	in-app	`standard.confirmStage`

UI:

- `/notifications` — список з фільтрами по типу і read/unread, групуванням по даті
- Іконка-дзвіночок у топбарі з лічильником непрочитаних
- `/admin/settings` (тільки ADMIN) — тумблери для всіх типів + lead-times

5.9 Журнал активності (audit log)

Кожна мутація → `ActivityLog`. Поля: `userId`, `action`, `entity`, `entityId`, `before`, `after`, `note`, `createdAt`. UI: `ActivityFeed` компонент який показує дії з диффом (computed client-side, fallback на сервер).

Покриття: статичний аналіз `pnpm test:audit-coverage` сканує всі tRPC-мутації, очікує що кожна містить виклик `logActivity(`. Allow-list для тривіальних UX-мутацій (markRead, тощо). В CI як required step.

5.10 Адміністрування

- `/admin/users` — список усіх користувачів, зміна глобальної ролі, активація/деактивація, скидання пароля, перегляд РГ-членств. Тільки ADMIN.
- `/admin/settings` — системні налаштування сповіщень.

5.11 Дашборд

Головна сторінка `/dashboard`. KPI-картки (стандартів активних, засідань цього місяця, моїх завдань, непрочитаних сповіщень, протоколів очікують). Найближчі засідання (правий рейл). Прострочені етапи. Активність останніх 7 днів.

5.12 Тех-стек і архітектура

Stack:

- **Frontend:** Next.js 14 App Router + React 18 + TypeScript + Tailwind CSS
- **API:** tRPC v11 + Zod
- **ORM:** Prisma 5 → Postgres 16+
- **Auth:** NextAuth Credentials + JWT
- **Email:** Resend (рекомендовано) або SMTP fallback
- **Storage:** S3-compatible (AWS S3, R2, B2, MinIO для self-host)
- **PDF/Word:** `@react-pdf/renderer` + `docx`
- **Testing:** Vitest для unit, `scripts/test-modules.ts` для інтеграційних

Архітектурні правила:

1. Уся бізнес-логіка — на сервері. Клієнт ніколи не обчислює дозволи; UI читає `can()` тільки для приховування кнопок, але сервер re-check'ає.
2. Усі мутації — `protectedProcedure` з RBAC всередині.
3. Жодних raw SQL крім обережних агрегацій (analytics).
4. `prisma.standard.findUnique(...)` мовою БД зрозумілий; в коді тримати схему single source of truth.

Деплой:

- Docker multi-stage (Node 20 bookworm-slim). Образ standalone, працює на будь-якому Docker-хості.
- Postgres 16+ обов'язковий. Бажано managed (Railway/Neon/Supabase).
- S3 опційно, але потрібен для документів і бекапів.

5.13 Безпека

- Bcrypt rounds=12 для паролів.
- NextAuth JWT, secret з env (≥32 байти).
- Усі мутації — POST через tRPC, CSRF-protected by design (same-origin).
- Усі `Authorization: Bearer` для cron — opaque secret від env.
- Усі вивантаження документів — через підписані URL до S3.
- Жодних паролей / API-ключів у логах. Усі env-секрети валідуються через Zod на старті.

---

6. ПОКРАЩЕННЯ (Phase 2) — quality of life

Без цього база працює. Це додає швидкість/комфорт/масштабованість.

6.1 Mobile responsive

- Hamburger drawer замість бокового сайдбару на `<lg`.
- Modal-ки стають bottom-sheet на `<md`.
- Усі таблиці приховують неосновні колонки на mobile (`hidden md:table-cell`).

- Сторінки списків (тасків, засідань) стекаються вертикально на `<lg`.
- Touch-targets ≥44px на тапабельних кнопках.
- iOS Safari fix: `body overflow:hidden` + `h-[100dvh]` на app shell — топбар прибитий до верху, не зникає при auto-hide URL bar.

6.2 Глобальний пошук Cmd+K

Команд-палет у центрі екрану, відкривається `Cmd/Ctrl+K`. Шукає по стандартах, засіданнях, завданнях, РГ, **людях**. Клавіатурна навігація: ↑↓ між результатами, Enter → відкрити, Esc → закрити. Топбар має кнопку «Пошук» що теж відкриває палет.

6.3 .ics експорт + календарна підписка

- Кнопка «📅 Календар» на засіданні → завантажує `.ics` файл для Outlook/Google/Apple Calendar.
- На сторінці РГ — кнопка «Підписатися» → копіювана URL `/api/working-groups/<id>/ical?user=<id>&token=<hmac>`. Календарні застосунки можуть підписатися; URL стабільний, валідується HMAC-токеном (з `NEXTAUTH_SECRET`), оновлюється раз на годину.

6.4 Mass-actions

На `/standards` (і за аналогією на `/tasks`, `/users`): чекбокси у таблиці, sticky bottom bar з діями «Змінити статус», «Перенести в РГ», «Архівувати». Per-row permission check, скіпнуті повертаються в response.

6.5 Аналітика для керівництва

Окрема вкладка в `/reports` тільки для DIRECTOR/ADMIN. KPI: усього в плані / в графіку / з простроченням / готові / завершено квартал. Діаграми (recharts):

- Розподіл по поточному етапу (bar)
- Виконано етапів за 6 місяців (bar) Таблиця по РГ з % готовності і кількістю засідань 30 днів. Top-5 найризикованіших стандартів.

6.6 DnD reorder у редакторі протоколу

`@dnd-kit/sortable`. Кожен пункт має grip-handle (≡). Перетягування секції перенумеровує `order` і помічає рядки dirty. Зберігається через єдиний «Зберегти все».

6.7 Тижневий дайджест (Пн 09:00 Київ)

In-process `node-cron` (через `src/instrumentation.ts`). По кожній РГ — окремий email керівнику/заступнику/секретарю з: простроченими етапами, дедлайнами тижня, засіданнями тижня, активними голосуваннями. DIRECTOR/ADMIN отримує всеохоплюючий звіт.

6.8 @mentions у коментарях

В textarea — `@` тригерить dropdown членів РГ з фільтром. На вибір вставляється `@[Ім'я](userId)`. При збереженні backend парсить це і шле in-app сповіщення згаданим.

6.9 Експорт документів

- `/api/meetings/[id]/protocol.docx` — формальний Word протокол з усіма реквізитами.
- `/api/meetings/[id]/protocol` — PDF.
- `/api/reports/plan.docx` і `plan.pdf` — звіт «Поетапний план виконання» (landscape).

6.10 Авто-бекапи бази

Cron 03:00 Київ → `pg_dump | gzip | aws s3 cp` → бакет S3, ретеншн 30 днів. Усе в тому самому Next.js процесі (через `instrumentation.ts`), без зовнішнього scheduler-а. Pg_dump версія має співпадати з сервером (зараз 18).

6.11 Темна тема

Toggle у топбарі. Зберігається в `localStorage`. Усі кольори через CSS-змінні (`--c-card`, `--c-ink`, `--c-page`, `--c-hairline`). Tailwind dark mode = class strategy на `<html>`.

6.12 Multi-tenant / Self-hosted

Образ підіймається з `docker compose --profile fullstack up -d` — підіймає app + Postgres + MinIO (S3) + Redis. Усе працює з `.env.example` без додаткових кроків. Документувати в README.

---

7. Нефункціональні вимоги

Параметр	Цільове значення
Час логіну → дашборд	≤ 2 сек на середньому 4G
Час відповіді tRPC-query	p95 ≤ 300 мс
Розмір прод-образу	≤ 250 MB
Підтримка одночасних користувачів	≥ 50 (1 instance), легке масштабування до 500
Відновлення з бекапу	≤ 5 хв
Час уптайму	99.5% (Railway / Fly.io контракт)
i18n	Українська як основна, інтерфейс не «hard-code'ить» англійську
Доступність	WCAG 2.1 AA для основних flow (логін, навігація, форми)

---

8. Послідовність розробки

Рекомендовано:

1. **Sprint 1 (1–2 тижні):** Схема + Auth + RBAC + Users + WG + базовий dashboard.
2. **Sprint 2 (1–2 тижні):** Standards CRUD + lifecycle + stage tracking + StandardProgress UI.
3. **Sprint 3 (1–2 тижні):** Meetings + Attendance + Protocol editor.
4. **Sprint 4 (1 тиждень):** Tasks + Comments + Voting.
5. **Sprint 5 (1 тиждень):** Notifications (backend + cron в-process) + Audit log + admin/settings.
6. **Sprint 6 (1 тиждень):** Export (Word/PDF) + Reports (program plan).

База готова за ~6–9 тижнів силами 2 інженерів.

Покращення (Phase 2) — далі по 1 тижню на епік (mobile, search, calendar, mass-actions, analytics, DnD, mentions, backups). Усього ~8 тижнів.

---

9. Чого свідомо НЕМАЄ в системі

Перерахування пастки для скоупу:

- Електронний підпис документів (КЕП через Diia.Pidpys) — не в Phase 1/2, можна як Phase 3.
- Friendly-URL для стандартів (slug-based) — стандарти живуть на cuid, не критично.
- Granular permissions «field-level» — нема. РБАС працює на рівні entity+action.
- Білінг / платежі — це внутрішня система, не SaaS.
- WebSocket-real-time notifications — наразі polling (60 сек) + page refetch. Достатньо для use-case.
- Mobile native app (iOS/Android) — тільки PWA-сумісний responsive web.

Кінець ТЗ.